

\$ git init - VERSION CONTROL

Git & GitHub 입문 가이드

버전 관리가 처음이어도 괜찮습니다. 핵심 개념부터 실전 명령어·협업·고급 기능까지 한 번에.

11개 챕터 요약

입문자를 위한

CONTENTS

학습 여정

01 버전 관리 & Git 기초

03 GitHub & 원격 저장소

05 베스트 프랙티스 & 오픈소스

02 브랜치와 병합

04 GitHub 협업

06 고급 기능 & 트러블슈팅

01

버전 관리 & Git 기초

Version Control & Git Basics

CHAPTER 01

버전 관리란?

파일의 변경 이력을 시간순으로 기록하고 추적하는 시스템. `보고서_최종_진짜최종_v2` 의 혼란을 끝냅니다.

이력 추적

모든 변경을 스냅샷으로, 언제든지 비교·복원

협업

여러 사람이 충돌 없이 동시에 작업

안전망

마음껏 실험하고 잘못되면 되돌리기

CHAPTER 01

Git vs GitHub

TOOL

Git

내 컴퓨터에서 돌아가는 **버전 관리 도구**. 커밋·브랜치·병합의 핵심 엔진.

PLATFORM

GitHub

Git 저장소를 **클라우드에 호스팅**. 협업·코드 리뷰·이슈·PR 기능 제공.

CHAPTER 02

변경이 흐르는 3개의 영역

작업 디렉터리

Working Directory

git add



스태이징 영역

Staging Area

git commit



저장소

Repository

```
$ git init
```

```
$ git add .
```

```
$ git commit -m "첫 커밋"
```

첫 커밋

02

브랜치와 병합

Branching & Merging

CHAPTER 03

브랜치 만들고 합치기

브랜치 다루기

```
$ git switch -c feature  
$ git switch main  
$ git merge feature
```

git log --graph

```
*7f3a1c (main) 병합  
|\n| * f4e5d6 품 추가  
|/  
*9f8e7d 초기 커밋
```


CHAPTER 03

병합 충돌 해결

충돌은 오류가 아니라 정상적인 협업 과정. Git이 표시한 부분을 정리한 뒤 다시 커밋합니다.

conflict markers

<<<<<< HEAD

const title = "내 변경";

=====

const title = "상대 변경";

>>>>>> feature

원하는 코드만 남기고

\$ git add . && git commit

03

GitHub & 원격 저장소

GitHub & Remote Repositories

CHAPTER 04

GitHub 시작 3단계

01

계정 만들기

github.com에서 무료 가입

02

인터페이스 익히기

Repositories · Issues · Pull requests · Actions

03

첫 저장소 만들기

New repository → README · .gitignore 추가

CHAPTER 05

원격 저장소 다루기

연결의 기본 별칭은 `origin` — 올릴 땐 push, 받을 땐 pull.

bash

```
$ git remote add origin <url>
```

```
$ git push -u origin main # 올리기
```

```
$ git pull # 받기
```

```
$ git clone <url> # 통째로 복제
```

04

GitHub 협업

Collaborating on GitHub

CHAPTER 06

기여의 표준 흐름



Fork 원본을 내 계정으로 복사

Issue 버그·요청·할 일 기록

PR "합쳐 주세요" 제안 + 코드 리뷰

권한 Read → Write → Admin 단계

05

베스트 프랙티스 & 오픈소스

Best Practices & Open Source

CHAPTER 07

좋은 커밋 메시지

× 나쁜 예

"수정"

"asdf"

"버그 고침 (또)"

✓ 좋은 예

"로그인 유효성 검사 추가"

"결제 반올림 오류 수정"

제목은 **50자 이내·명령형**으로 "무엇을 왜". 커밋 하나엔 논리적 변경 하나.

CHAPTER 08

대표 라이선스 한눈에

MIT

가장 자유로움. 저작권 표시만 유지하면 상업적 사용·수정·재배포 자유.

Apache 2.0

MIT와 비슷하되 특허권 관련 보호 조항 추가.

GPL

카피레프트. 이 코드를 쓴 결과물도 같은 라이선스로 공개해야 함.

06

고급 기능 & 트러블슈팅

Advanced Features & Troubleshooting

CHAPTER 09

알아두면 좋은 4가지

git stash

작업을 잠시 치워 두고 나중에 되돌리기

git rebase

이력을 직선으로 정리 **(공유 브랜치 금지)**

git hooks

이벤트 발생 시 자동 실행 스크립트

blame / bisect

누가 바꿨나 추적 · 버그 커밋 찾기

CHAPTER 10

당황하지 마세요

실수로 reset하거나 브랜치를 지워도 `reflog` 가 거의 모든 기록을 갖고 있습니다.

사라진 커밋 되살리기

```
$ git reflog # 모든 이동 기록
```

```
$ git checkout a1b2c3d # 잃은 커밋으로 복귀
```

CHEAT SHEET

명령어 치트시트

시작

```
git init  
git clone <url>
```

커밋

```
git add .  
git commit -m "..."
```

이력

```
git status  
git log --oneline
```

브랜치

```
git switch -c <n>  
git merge <n>
```

원격

```
git pull  
git push
```

되돌리기

```
git revert <c>  
git stash pop
```


NEXT STEPS

이제, 매일 커밋하세요

- 01 작은 개인 프로젝트를 GitHub에 올려 매일 커밋
- 02 오픈소스의 `good first issue` 하나 해결
- 03 막히면 공식 문서·커뮤니티를 적극 활용



AI 정보공유 채팅방 참여 문의

open.kakao.com/o/sv3aJD4h

```
$ git push origin main - happy committing
```